



MÓDULO PROYECTO

CFGS Desarrollo de Aplicaciones WEB
Informática y Comunicaciones

AJCAR25

Tutor individual: José Antonio Carrascal Alderete

Tutor colectivo: José Antonio Carrascal Alderete

Año: 2026

Fecha de presentación: 29/05/2026

Nombre y Apellidos: Héctor Sanz Riesco
Email: hsanzriesco@gmail.com

**Foto actual
del alumno**

Tabla de contenido

1	Identificación proyecto	3
2	Organización de la memoria	3
3	Descripción general del proyecto	3
3.1	Objetivos.....	3
3.2	Cuestiones metodológicas	4
3.3	Entorno de trabajo (tecnologías de desarrollo y herramientas)	4
4	Descripción general del producto	4
4.1	Visión general del sistema: límites del sistema, funcionalidades básicas, usuarios y/o otros sistemas con los que pueda interactuar.	4
4.2	Descripción breve de métodos, técnicas o arquitecturas(m/t/a) utilizadas.....	5
4.3	Despliegue de la aplicación indicando plataforma tecnológica, instalación de la aplicación y puesta en marcha	5
5	Planificación y presupuesto	6
6	Documentación Técnica: análisis, diseño, implementación y pruebas.	6
6.1	Especificación de requisitos	6
6.2	Análisis del sistema	7
6.3	Diseño del sistema:	8
6.3.1	Diseño de la Base de Datos	8
6.3.2	Diseño de la Interfaz de usuario.....	8
6.3.3	Diseño de la Aplicación.	9
6.4	Implementación:	9
6.4.1	Entorno de desarrollo.	9
6.4.2	Estructura del código.	10
6.4.3	Cuestiones de diseño e implementación reseñables.	10
6.5	Pruebas.	11
7	Manuales de usuario	11
7.1	Manual de usuario	11

7.2	Manual de instalación	11
8	Conclusiones y posibles ampliaciones	11
9	Bibliografía	12
10	Anexos	12

1 Identificación proyecto

AJCAR25 es una aplicación web desarrollada para digitalizar la gestión integral de un taller de chapa y pintura. Centraliza presupuestos, vehículos en taller, facturación automática, control de stock y rendimientos de los empleados en una única plataforma accesible desde cualquier dispositivo.

2 Organización de la memoria

La memoria se ha organizado siguiendo el esquema oficial del TFG, siguiendo los siguientes apartados:

1. En la identificación del proyecto explicamos el proyecto y sus objetivos principales.
2. Describimos la estructura del documento de la memoria y el contenido de cada apartado.
3. Explicamos los objetivos del proyecto, la metodología de trabajo seguida y las tecnologías y herramientas utilizadas durante el desarrollo de este.
4. Detallamos la visión general del proyecto, los tres roles de usuario, jefe y empleado, la arquitectura de la aplicación y el proceso de despliegue en producción.
5. Planificación temporal del desarrollo del proyecto dividida en fases y una estimación del presupuesto del proyecto.
6. Explicamos la especificación de requisitos, el análisis del sistema, el diseño de la base de datos, el diseño de la interfaz, la estructura del código, los aspectos mas reseñables de la implementación y las pruebas realizadas.
7. Dos manuales, el manual del usuario con las instrucciones de uso de cada panel y el manual de instalación para instalar el proyecto en local.
8. Reflexión sobre los objetivos conseguidos, las dificultades del proyecto y las posibles ampliaciones.
9. Fuentes consultadas sobre el desarrollo del proyecto.
10. Variables de entorno necesarias y las credenciales de acceso para la evaluación.

3 Descripción general del proyecto

3.1 Objetivos

1. Digitalizar la solicitud y gestión de presupuestos del taller eliminando el uso del papel.
2. Proporcionar a los clientes un área personal con estados de sus vehículos en tiempo real.
3. Ofrecer a los empleados un panel de trabajo para gestionar el flujo diario de forma eficiente.

4. Dar al jefe visión global del negocio: ingresos, stock y rendimiento del equipo.
5. Automatizar el envío de presupuestos y facturas en PDF por correo electrónico.
6. Garantizar la seguridad con autenticación y contraseñas cifradas.

3.2 Cuestiones metodológicas

Para desarrollar este proyecto seguí una metodología iterativa e incremental. En lugar de planificar todo desde el principio, fui añadiendo funcionalidades poco a poco y probando cada una antes de continuar.

Las fases principales fueron: análisis y diseño del sistema, configuración del entorno de desarrollo, desarrollo del backend (APIs y base de datos), desarrollo del frontend (paneles de usuario), integración y pruebas, mejoras iterativas y finalmente la documentación.

Como herramienta de control de versiones usé Git con GitHub, haciendo commits frecuentes. Gracias a la integración con Vercel, cada push a la rama principal desplegaba automáticamente la nueva versión en producción, lo que me permitió probar los cambios en un entorno real desde el principio.

3.3 Entorno de trabajo (tecnologías de desarrollo y herramientas)

FrontEnd:

- Next.js 14 + React 18 + TypeScript
- TailWind CSS
- jsPDF + jsPDF-autotable

Backend y Base de Datos:

- Neon (PostgreSQL)
- bcryptJS (Cifrado de contraseñas)

Herramientas de desarrollo:

- VSCode + Git + GitHub + Vercel (Despliegue) + Neon Console

4 Descripción general del producto

4.1 Visión general del sistema: límites del sistema, funcionalidades básicas, usuarios y/o otros sistemas con los que pueda interactuar.

El sistema define tres roles de usuario con permisos diferenciados que cubren todo el ciclo de negocio:

- Cliente:

- Solicitar presupuestos, consultar el estado de sus vehículos en tiempo real, descargar facturas en PDF, ver mantenimientos cancelados con motivo y modificar datos personales y contraseña.
- Empleado:
 - Gestionar presupuestos pendientes, ingresar vehículos aceptados al taller, generar facturas con actualización automática de stock, crear fichas de clientes y consultar almacén.
- Jefe:
 - Dashboard con ingresos totales y del mes, gestión de empleados y clientes, stock con entrada masiva, facturas filtradas por matricula, objetivos mensuales y ranking de facturación por empleado.

Adicionalmente existe una página publica con información del taller y formulario de presupuesto accesible sin registro ninguno.

4.2 Descripción breve de métodos, técnicas o arquitecturas(m/t/a) utilizadas.

- Arquitectura con Next.js App Router: frontend y backend en el mismo proyecto sin servidores separados.
- API REST con route handlers por recurso (/api/presupuestos, api/facturas...)
- Refresco automático de la página cada 5 segundos en los paneles para refrescar datos silenciosamente sin recargar la página.
- Generación de PDFs en el navegador, conversión a Base64 y envío adjunto al servidor para el email (NodeMailer).
- Autenticación con SessionStorage y contraseñas cifradas con bcryptJS.
- Validación de contraseñas tanto como en el frontend tanto en el backend.

4.3 Despliegue de la aplicación indicando plataforma tecnológica, instalación de la aplicación y puesta en marcha

- Para el despliegue del proyecto he utilizado Vercel + Neon con PostgreSQL para la base de datos. Utilizamos variables de entorno en vercel (DATABASE_URL para Neon, EMAIL_USER Y EMAIL_PASS para el email).
- Para ejecutar el proyecto en local se necesita Node.js 18+ y una cuenta Gmail con contraseña de aplicación. La base de datos ya está alojada y operativa en Neon, por lo que no requiere instalación ni configuración adicional. Los pasos son:
 1. Clonar el repositorio desde GitHub, (git clone <https://github.com/hsanzriesco/ajcar25.git>).
 2. Ejecutar npm install para instalar todas las dependencias.
 3. Crear el archivo .env.local en la raíz del proyecto con las variables DATABASE_URL (cadena de conexión de Neon), EMAIL_USER y EMAIL_PASS.

4. Arrancar el servidor de desarrollo con `npm run dev` y acceder desde el navegador en `http://localhost:3000`. El servidor recargará automáticamente los cambios en caliente sin necesidad de reiniciarlo

5 Planificación y presupuesto

5.1 *planificación temporal (+/- 4 meses)*

Semanas	Descripción
Semana 1-2	Análisis y diseño: requisitos, modelo de datos, bocetos de interfaz.
Semana 3	Configuración del entorno, esquema de tablas, despliegue inicial en vercel.
Semana 4-6	Desarrollo del backend: APIs, autenticación, lógica de negocio y emails.
Semana 7-10	Desarrollo del frontend: página principal, formularios y tres paneles de usuario.
Semana 11-12	Integración completa, pruebas funcionales y corrección de errores.
Semana 13-15	Mejoras iterativas: validación, filtros y estilo del panel cliente.
Semana 16	Redacción de la memoria y documentación.

5.2 *Presupuesto*

El proyecto se ha desarrollado íntegramente con herramientas gratuitas: Next.js, React, Tailwind, Vercel, Neon, Gmail. El coste real de producción es 0€/mes para el volumen actual del taller.

6 Documentación Técnica: análisis, diseño, implementación y pruebas.

6.1 *Especificación de requisitos*

1. Registro y autenticación de usuarios por rol (cliente, empleado y jefe) con redirección automática al panel correspondiente.
2. Solicitud de presupuestos desde la web publica sin registro o desde el panel del cliente con datos precargados.

3. Gestión de facturas y presupuestos en PDF con envío automático al cliente por email.
4. Actualización automática del stock de artículos al facturar.
5. Panel de rendimiento con objetivos mensuales y ranking de facturación por empleado.
6. Recuperación de contraseña por email con token temporal con fecha de expiración.
7. Actualización silenciosa de datos en los tres paneles cada 5 segundos.
8. Filtrado de facturas por matrícula del vehículo o nombre del cliente.
9. Protección de rutas por rol: acceso a panel incorrecto cierra la sesión y redirige al login.

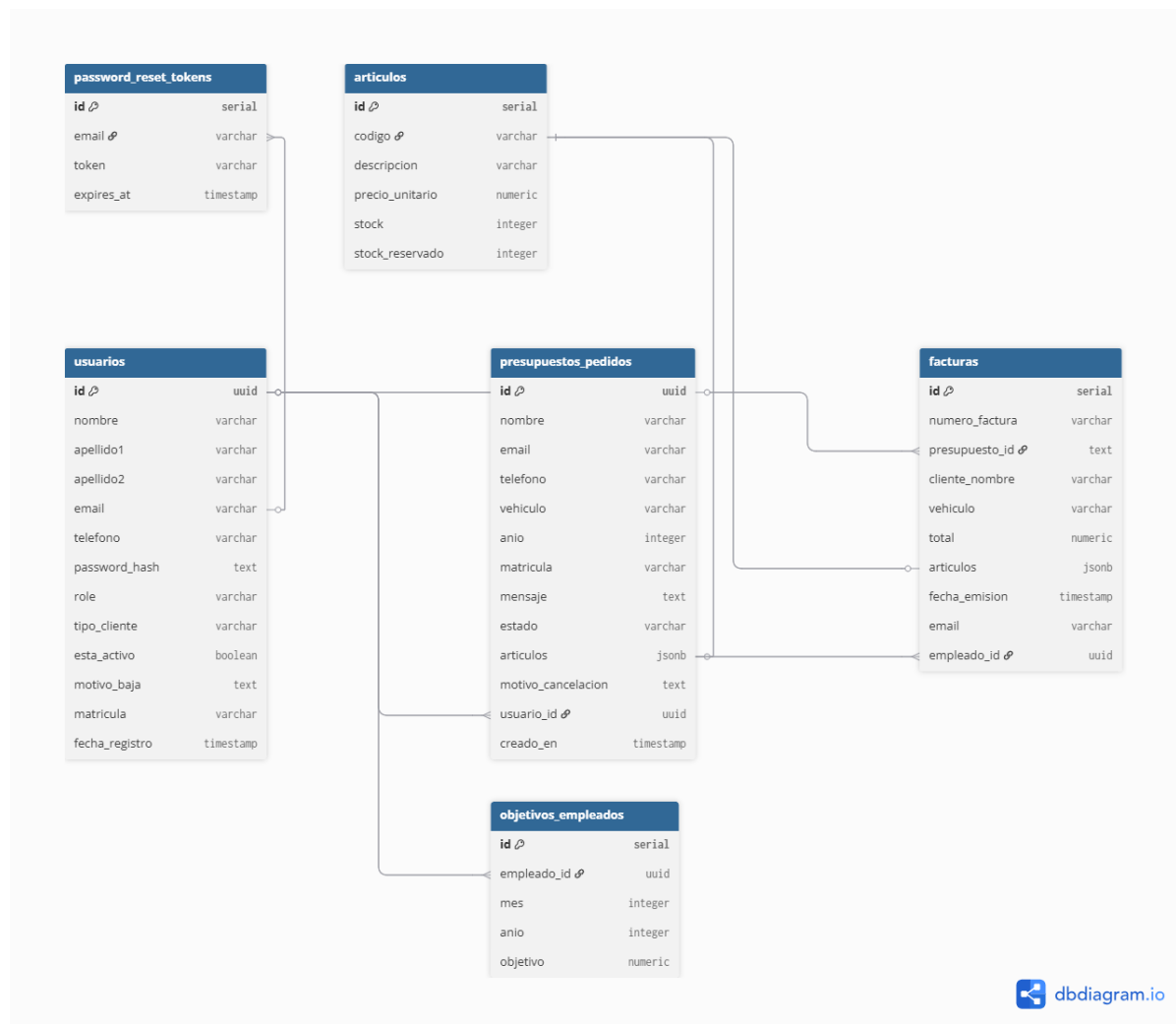
6.2 *Análisis del sistema*

El sistema cuenta con tres actores:

1. El cliente solicita presupuestos, consulta el estado de sus vehículos en tiempo real, descarga facturas en PDF y gestiona su perfil.
2. El empleado gestiona el flujo de presupuestos, añade artículos, envía PDFs al cliente, ingresa vehículos al taller, genera facturas y crea fichas de nuevos clientes verificando su DNI/CIF.
3. El jefe tiene acceso total, gestión de empleados con matrícula autogenerada, control de clientes, stock con entrada masiva, consulta de todas las facturas y seguimiento del rendimiento del equipo con objetivos mensuales.

6.3 Diseño del sistema:

6.3.1 Diseño de la Base de Datos



6.3.2 Diseño de la Interfaz de usuario.

El diseño de la interfaz del usuario lo he ejecutado con figma, tenemos la carpeta en: Documentacion/diseño_de_interfaz/interfaces.zip.

6.3.3 Diseño de la Aplicación.



6.4 Implementación:

6.4.1 Entorno de desarrollo.

Como entorno de desarrollo he utilizado Node.js 18, npm para las dependencias, Next.js 14 con TypeScript estricto y VSCode para la escritura de los códigos. Para el despliegue he utilizado Git y GitHub con despliegue automático en Vercel, en cada push apunta al main. Variables de entorno en env.local para el desarrollo local.



6.4.2 Estructura del código.

El proyecto sigue la estructura del App Router de Next.js 14. Todas las páginas están dentro de la carpeta `app/`, organizadas por ruta. Por ejemplo, la página del panel del cliente está en `app/cliente/page.tsx` y el panel del empleado en `app/gestion/tareas/page.tsx`.

Los endpoints de la API están dentro de `app/api/`, cada uno en su propia carpeta con un archivo `route.ts`. Por ejemplo, toda la lógica de los presupuestos está en `app/api/presupuestos/route.ts` y la de las facturas en `app/api/facturas/route.ts`.

Los componentes reutilizables como el formulario de login, el de registro, el de presupuesto y el selector de teléfono están en la carpeta `components/` en la raíz del proyecto.

Las imágenes estáticas (logo, fondos y fotos del taller) están en `public/imagenes/`.

Los tres paneles principales (cliente, empleado y jefe) son páginas de una sola vista que gestionan múltiples secciones mediante estado interno con `useState`, sin recargar la página al cambiar de pestaña.

6.4.3 Cuestiones de diseño e implementación reseñables.

PhoneInput propio En lugar de usar librerías externas, desarrollé mi propio componente de selección de teléfono con 195 países, buscador y validación por longitud. Así eliminé dos dependencias del proyecto y tuve control total sobre el comportamiento.

Generación de PDFs en el navegador Las facturas se generan directamente en el navegador con `jsPDF`, se convierten a Base64 y se envían al servidor solo para adjuntarlas al email. De esta forma no es necesario guardarlas en ningún sitio del servidor.

JOIN con cast para la matrícula La tabla facturas no guarda la matrícula para no duplicar datos. Para obtenerla hago un LEFT JOIN con `presupuestos_pedidos` usando `f.presupuesto_id::uuid = p.id`, ya que `presupuesto_id` es de tipo TEXT y `p.id` es UUID, y sin el cast PostgreSQL da error.

Protección de rutas por rol Antes de renderizar cada panel verifico el rol guardado en `sessionStorage`. Si no coincide, limpio la sesión y redirijo al login. Para evitar que el contenido aparezca un instante antes de la redirección, uso un estado autorizado que empieza en `false` y solo se pone a `true` cuando el rol es correcto.

Polling automático Los tres paneles se actualizan cada 15 segundos con un `setInterval` que hace `fetch` en segundo plano sin mostrar ningún spinner. Cuando el componente se desmonta, el `clearInterval` del `useEffect` evita que sigan ejecutándose peticiones innecesarias.

Validación de contraseñas en dos capas Valido la contraseña tanto en el frontend (indicador visual en tiempo real y botón deshabilitado) como en el backend (antes de tocar la base de datos). Así, aunque alguien llame directamente a la API sin pasar por el formulario, la validación sigue activa.

6.5 Pruebas.

Autenticación y autorización:

- Login con credenciales correctas e incorrectas para los tres roles. Acceso por URL a rutas protegidas con rol incorrecto. Flujo completo de recuperación de contraseña con token válido.

Flujo de presupuesto y facturación:

- Solicitud de presupuesto desde web pública y panel cliente. Envío de PDF al cliente, aceptación y rechazo. Ingreso al taller, facturación y verificación de actualización de stock. Cancelación con motivo obligatorio.

Seguridad y responsividad:

- Intento de contraseñas débiles directamente en las APIs (validación del backend activa). Verificación de cifrado en base de datos.

7 Manuales de usuario

7.1 Manual de usuario

El manual del usuario se podrá visualizar en la carpeta

Documentacion/Manual_De_Usuario/Manual_de_Usuario.pdf.

7.2 Manual de instalación

El manual de instalación se podrá visualizar en la carpeta

Documentacion/Manual_de_Instalacion/manual_de_instalacion.pdf

8 Conclusiones y posibles ampliaciones

El proyecto cumple todos los objetivos planeados: es una aplicación web completa, funcional y desplegada en producción que digitaliza la gestión integral de un taller de chapa y pintura. Durante el desarrollo se han consolidado conocimientos de React/TypeScript, API REST con Node.js, bases de datos relacionales PostgreSQL, despliegue cloud y seguridad en aplicaciones web.

Posibles ampliaciones futuras:

- Notificaciones en tiempo real.
- Galería fotográfica del vehículo.
- Firma digital de aceptación de presupuestos y facturas para mayor seguridad jurídica.
- Estadísticas avanzadas en el dashboard del jefe: gráficos de ingresos por mes y tipos de reparación.
- Aplicación móvil.

9 Bibliografía

- [Next.js](#)
- [React](#)
- [TailWind Css](#)
- [Neon](#)
- [jsPDF](#)
- [Nodemailer](#)
- [Bcryptjs](#)
- [Vercel](#)
- [MDN Web](#)
- [TypeScript](#)

10 Anexos

1. Variables de entorno:
Para ejecutar el proyecto es necesario configurar las siguientes variables de entorno en el archivo env.local:
 - **DATABASE_URL:**
postgresql://neondb_owner:npg_oIY8Xn2idmLG@ep-sparkling-math-aid6fnxf-pooler.c-4.us-east-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require
 - **EMAIL_USER:** ajcar391@gmail.com
 - **EMAIL_PASS:** yndayqhomxjocrwq
2. URL de producción:
 - La aplicación está desplegada y accesible públicamente con esta URL:
<https://ajcar25.vercel.app>
3. Repositorio de GitHub:
 - El código fuente completo esta disponible en:
<https://github.com/hsanzriesco/ajcar25>
4. Credenciales de evaluación:
 - Las credenciales de todos los usuarios con todos los roles son los siguientes:
 - Jefe:
 - jefe@gmail.com y/o 594859
 - 123456Aa&
 - Empleado:
 - empleado@gmail.com y/o 433736
 - 123456Aa&
 - Usuario:
 - hsanzriesco@gmail.com
 - 123456Aa&